EJERCICIO 1

Se debe implementar un algoritmo para Java 21 que permita obtener el valor minimo de un vector de numeros enteros. El algoritmo debe tomar el primer valor del vector y recorrerlo hasta el final mediante un bucle for. Al finalizar el recorrido, debe mostrar o retornar el resultado con el siguiente formato: Valor minimo del vector: (valor). Con este metodo indicado, se espera una complejidad espacial de O(1) y temporal de O(n). Ordenar el arreglo para luego buscar el numero mas pequeño significaria aumentar la complejidad temporal a O(n log n) como minimo. El codigo debe incluir manejo de excepciones para los casos extremos, como por ejemplo donde el vector sea nulo o este vacio. Se requiere que el codigo sea comentado de forma clara y tambien debe indicar porque el algoritmo elegido es el mas eficiente.

EJERCICIO 2

Se debe implementar un algoritmo para java 21 que permita buscar un elemento dentro de un vector de numeros enteros desordenado. Se requiere que sea implementado mediante una busqueda lineal sin ordenar previamente los elementos del vector. Esta busqueda se debe realizar con un bucle for y debe detenerse en el momento que se encontro el elemento. Si se encuentra el elemento, debe retornar:
El elemento fue encontrado luego de recorrer (n) posiciones.
De no ser encontrado el numero porque no esta presente en el vector (siendo un vector no nulo ni vacio) debe retornar:
Se recorrieron todas las (n) posiciones y no se encontro el elemento.
Se espera una complejidad temporal de $\Omega(1)$ en el mejor caso y O(n) en los casos promedio y peor, y una complejidad espacial de O(1). El algoritmo debe poder realizar manejo de excepciones para casos extremos (por ejemplo. vector nulo o vacio) y debe comentar la logica del programa y justificar porque la estrategia elegida es la adecuada y analizar el mejor, peor y caso promedio.

EJERCICIO 3

Se requiere implementar un algoritmo para Java 21 que realice una busqueda binaria sobre un vector ordenado. Este algoritmo es el mas eficiente para encontrar un dato en un vector ordenado porque divide el problema a la mitad en cada paso (complejidad temporal O(log n)) en lugar de hacer una busqueda lineal cuya complejidad temporal es O(n). El código debe mostrar su funcionamiento paso a paso, imprimiendo en cada iteración la evolución de las variables de búsqueda (inicio, medio, fin), el valor evaluado en la posición central y el subvector o rango analizado hasta hallar el valor final (o determinar que no existe). Debe contemplar manejo de excepciones para casos extremos (vector nulo/vacio).

EJERCICIO 4

Se requiere la implementacion de dos algoritmos distintos para Java 21. Ambos deben poder detectar elementos duplicados en un vector.
El primer algoritmo debe realizarse mediante dos bucles anidados. El segundo debe emplear una estructura HashSet para resolver el problema. Se requiere manejo de excepciones (vector nulo/vacio) y que se comente un analisis detallado sobre la

comparacion de las complejidades temporal y espacial de cada algoritmo y el trade off que realiza cada uno.

EJERCICIO 5

Se requiere la implementacion de un algoritmo para java 21 que cuente cuantas veces aparece un elemento dentro de un vector de numeros enteros. El algoritmo debe recorrer el vector hasta el final mediante un bucle for e incluir un contador que se incremente cada vez que el elemento a buscar aparece. Si no se recorre el vector completo, hay una alta probabilidad de que el resultado final que devuelva el contador sea incorrecto. Si se hallo el elemento una o mas veces, debe retornar: El elemento fue hallado n veces. De lo contrario, debe retornar: El elemento no fue hallado en el vector. Debe permitir manejo de excepciones para casos extremos (vector nulo/vacio) y el codigo debe ser comentado con un analisis sobre su complejidad temporal y espacial.

EJERCICIO 6

Se requiere la implementacion de un algoritmo para java 21 que pueda determinar si dos vectores son iguales. Se deben recorrer simultaneamente ambos vectores con un bucle for y, en el momento que los valores de la misma posicion son distintos entre si, se debe cortar el bucle inmediatamente mediante un return false. Si ambos vectores son iguales, se debe devolver true. Se solicita que se comente un analisis completo sobre las complejidades espacial y temporal (caso promedio, mejor caso y peor caso).

EJERCICIO 7

Se requiere la implementacion de dos algoritmos para java 21. Ambos deben poder invertir un vector.
El primer algoritmo debe ser implementado mediante el uso de un vector auxiliar donde se vayan copiando los elementos del vector original pero desde la posicion n-1 hasta la posicion 0.
El segundo algoritmo debe invertir el vector mismo sin utilizar memoria extra significativa (no debe crear otro vector auxiliar). En su lugar, debe utilizar dos punteros.
El algoritmo debe poder manejar excepciones sobre vector vacio/nulo y debe comentar una comparacion que indique ventajas y desventajas de cada uno de los algoritmos, el trade off que hace cada uno y la complejidad temporal y espacial que tienen.

EJERCICIO 8

Se requiere implementar un algoritmo para java 21 que calcule la suma de todos los elementos de una matriz. El algoritmo debe recorrer la matriz con dos bucles for anidados, e ir sumando elemento a elemento en una variable auxiliar hasta llegar al final de la matriz. Además, debe incluir un contador que vaya sumando la cantidad de operaciones (sumas) que se realizan. Se requiere implementar manejo de excepciones en casos extremos como matriz vacia/nula y el codigo debe ser comentado con un analisis sobre las complejidades temporal y espacial.

EJERCICIO 9

Se debe implementar un algoritmo para Java 21 que encuentre el mayor elemento dentro de una matriz de numeros enteros. El algoritmo debe inicializar el valor maximo con el primer elemento de la matriz y recorrerla completa mediante dos bucles for anidados (uno para filas y otro para columnas). Antes del codigo debe justificar porque resulta necesario recorrer todos los elementos de la matriz para asegurar cual es el mayor, y solicitar el analisis de su complejidad temporal y espacial (se espera una complejidad temporal de O(n*m) siendo n filas y m columnas, y una complejidad espacial de O(1)). Al finalizar el recorrido debe retornar la salida con el siguiente formato: El mayor elemento de la matriz es: (valor) en la posicion [fila][columna]. El codigo debe incluir manejo de excepciones para casos extremos (matriz nula/vacia) y comentarios claros que expliquen la logica del programa.

EJERCICIO 10

Se requiere la implementacion en Java 21 del algoritmo de ordenamiento Bubble Sort para ordenar de menor a mayor un vector de numeros enteros. Antes del codigo, se debe explicar detalladamente porque este algoritmo resulta adecuado unicamente para fines didacticos o conjuntos pequeños de datos y porque resulta tan ineficiente para grandes volumenes de informacion debido a su complejidad temporal de O(n^2). El algoritmo debe ser implementado con bucles y debe incluir contadores para registrar tanto la cantidad de comparaciones como de intercambios realizados durante la ejecucion. Al terminar, la salida debe mostrar el vector ordenado y un resumen con el siguiente formato: Total de comparaciones realizadas: (cantidad). Total de intercambios realizados: (cantidad). El codigo debe contemplar manejo de excepciones para casos de vector vacio/nulo.